Shaders & VFX

Contents

- Post-Processing
  - How an effect is built
  - Renderer order
  - Resolution scaling
  - The effects
  - Adding an effect
- Dash Effects
- Impact Frame
- Object VFX
  - Dissolve
  - Shatter
  - MiddleOutFill (stagger bar)
  - Danger Volumes
- Volume Profiles
- Gotchas

The game's custom rendering:

- **Post-processing:** Sharpen, Quantised Dither, Downsample, Fear Vignette and CRT.
- **Dash Effects:** the ethereal phase-dash screen effect.
- **Impact Frame:** the Special Shot's full-screen impact.
- **Object VFX:** Dissolve, Shatter, the stagger bar fill and the Danger Volume telegraphs.

Everything targets **URP** on Unity 6 with the **Render Graph API** (Compatibility Mode off).

See also: Interface & Settings (Graphics menu, UI Render Target, resolution), Camera Effects, Fear Effects, Enemy Behaviour, Special Shot.

Post-Processing

How an effect is built

Each full-screen effect has three parts:

Part	Job
Shader	Reads the camera image (`_BlitTexture`) and writes the result.
Volume Component	Designer settings, added to a profile through **Add Override > Custom Post-Processing**.
Renderer Feature	On the **PC_Renderer** asset. Reads the Volume each frame and adds the render pass.

Each feature also has **Default Settings**, used for any parameter not overridden on the active profile. They're all set to "off" values on PC_Renderer, so an effect only shows when a profile turns it on.

Features only run on **Game** cameras with **Post Processing** ticked. Only one camera per image should have it ticked (PlayerCam for the world, the UI camera for the HUD), or effects apply twice.

Renderer order

The custom features on **PC_Renderer**, in list order:

#	Feature	Runs at
1	Sharpen	After Post Processing
2	Quantised Dither	After Post Processing
3	Downsample	After Post Processing
4	Fear Vignette	After Post Processing
5	Dash Effects	**Before** Post Processing
6	Impact Frame	After Post Processing
7	CRT	After Post Processing

Features at the same stage run in list order, so the image goes: URP's own effects (Bloom, grading) → Sharpen → Quantised Dither → Downsample → Fear Vignette → Impact Frame → CRT. Dash Effects runs before URP's effects, so bloom and grading apply on top of it.

CRT is last so its scanlines and vignette sit over everything, including the impact frame.

Resolution scaling

Effects are authored at **1080p**. The Render Resolution Manager sets `_ResolutionScale` (chosen height / 1080, so 2.0 at 4K), and anything measured in pixels is multiplied by it:

- Sharpen's sample distance
- CRT's blur offset and phosphor column width
- Downsample's block size (and so the dither cell size)

Effects that work across the screen (Dash Effects, Impact Frame, Fear Vignette, scanline count) don't need it.

The effects

Sharpen

Boosts each pixel against its four neighbours to bring out edges.

Setting	What it does
Sharpness	Strength. 0 = off.
Sample Distance	Neighbour distance in 1080p pixels.

Quantised Dither

Cuts the colours down to a set number of steps per channel, with an ordered dither to hide banding.

Setting	What it does
Red / Green / Blue Steps	Colour levels per channel.
Effect Strength	0 = off.
Dither Strength	0 = flat colour steps, no dither.
Bayer Size / **Use PS1 Matrix**	2×2 to 16×16 Bayer pattern, or the PS1's 4×4 pattern.
Use Perceived Brightness / **Perceptual Gamma**	Dithers by perceived brightness instead of linear.

Each dither cell matches a Downsample block, so every pixel block gets one flat dithered colour.

Downsample

Pixelates the image into square blocks. Each block takes one exact pixel from its centre, so blocks have flat colour and crisp edges.

Setting	What it does
Pixel Size	Block size in 1080p pixels. 1 = off.

Skips any camera named **UICam**, so the HUD isn't pixelated.

> **In code:** the feature works out the final block size in C# (`Pixel Size x _ResolutionScale`, rounded to whole pixels) and publishes it as the global `_DownsamplePixelSize`, which Quantised Dither uses as its cell size. If Downsample doesn't run for a camera, it publishes a fallback of the resolution scale rounded down (1 at 1080p and 1440p, 2 at 4K).

Fear Vignette

A vignette with a noisy edge that creeps inward. Driven by `FearPostProcessEffects` (see Fear Effects), which sets most of these values.

Setting	What it does
Vignette Intensity / Softness / Color	How far in, how soft, what colour.
Noise Intensity / Scale / Speed	How broken up the edge is and how fast it creeps.
Cycle Duration	How often each noise layer resets.
Blend Mode	0 Multiply, 1 Screen, 2 Overlay, 3 Hard Light.

> **In code:** two noise layers scroll inward half a cycle apart and cross-fade, so the creep never visibly stretches or pops.

CRT

Setting	What it does
Blur Offset	Soft blur. 0 = off.
Scanline Intensity / Count / Speed	Scrolling scanlines. Intensity 0 = off.
Vignette Intensity / Smoothness	Edge darkening.
Use Phosphor, **Phosphor Intensity / Scale**	RGB phosphor columns. Higher Scale reduces moire against the dither.

Adding an effect

1. **Shader:** a full-screen shader reading `_BlitTexture`. Multiply pixel distances by `_ResolutionScale`.
2. **Volume Component:** derive from `VolumeComponent`, add `[VolumeComponentMenu("Custom Post-Processing/Name")]`.
3. **Renderer Feature:** create the material in `Create()`, check for a Game camera with Post Processing in `AddRenderPasses`, destroy the material in `Dispose()`.
4. **Render Pass:** in `RecordRenderGraph`, read the Volume, set material values, then blit and copy back:

```
        TextureHandle src = resourceData.activeColorTexture;
        var desc = src.GetDescriptor(renderGraph);
        desc.name = "_MyEffectTexture";
        desc.depthBufferBits = 0;
        TextureHandle dst = renderGraph.CreateTexture(desc);

        renderGraph.AddBlitPass(new RenderGraphUtils.BlitMaterialParameters(src, dst,
material, 0), PassName);
        renderGraph.AddCopyPass(dst, src);
```

`RenderGraphUtils` needs `using UnityEngine.Rendering.RenderGraphModule.Util;`.

5. Add the feature to **PC_Renderer** in the right place, and the override to the scene profiles.
6. Optional: add it to the Graphics menu (see Interface & Settings).

Use the Sharpen files as a template. If the effect is usually off, give the Volume Component an `IsActive()` and skip the pass when it's false (Dash Effects and Impact Frame do this).

> **Use `material.SetFloat`, not `SetInt`.** Declare the shader property as `float` and cast to `int` inside the shader. `SetInt` doesn't reliably reach shaders, and once left a loop count at zero and caused a black screen.

Dash Effects

The dash is an ethereal phase dash: the player briefly slips into a ghost world. Four layers, each with its own toggle, all scaled by **Intensity**:

Layer	Look	Key settings
Ethereal Grade	Drains colour, tints shadows and highlights, glows bright areas, hazes the distance.	Desaturation, Shadow/Highlight Colour, Glow Threshold/Intensity, Depth Fade Start/End
Radial Blur	Zoom blur from the centre.	Sample Count, Blur Strength, Center Falloff
Warp	Stretches the screen outward (negative pulls in).	Warp Strength
Action Lines	Animated radial speed lines.	Lines Colour, Tiling, Remap (higher = fewer lines), Animation

A **Center Mask** keeps the middle of the screen clear of the blur and lines.

The Ethereal Grade runs first, so the other layers distort the already-graded image.

Runtime: `PlayerDash` fades **Intensity** up over **Dash Effects Fade In** when a dash starts and down over **Dash Effects Fade Out** after, multiplied by the player's Dash Effects setting. Leave Intensity at 0 on the profile.

> **In code:** the feature skips the pass entirely while Intensity is 0, and only asks URP for the depth texture while Depth Fade is on. The Ethereal Grade is shader pass 1, the other layers are pass 0.

Impact Frame

The Special Shot's anime-style impact, centred on the hit's position on screen. Four layers, switched on and off by `ImpactFrameController` (see Camera Effects):

Layer	Look
Flash	Solid colour. Overrides everything else.
Scene	The scene split into two flat colours, using brightness plus how much surfaces face upward.
Lines	Speed lines bursting from the centre. They invert the two-tone, or draw light colour over the normal scene.
Jitter	Slices of the image pushed outward.

Set the look on the Volume Component (leave the Layer toggles, Focal Point and Seed alone, they're set at runtime):

Setting	What it does
Dark / Light Colour, **Flash Colour**	The two tones, and which one the flash uses.
Scene Threshold	Higher = more of the scene goes dark.
Normals Weight	How much surface direction counts. Sprites don't have normals, so they split by brightness only.
Lines Tiling / Noise Scale / Threshold	How many lines, how broken up, how many survive.
Lines Clear Min / Max / Power	Clear area around the centre. Higher Power = mostly tight with a few long spikes.
Jitter Scale / Threshold / Strength	Slice count, how many move, how far.

> **In code:** ported from a reference Shader Graph, using the same noise hash as Shader Graph's Simple Noise so patterns match. The feature only runs while a layer is on, and only then asks

> URP for camera normals.

Object VFX

Dissolve

Burns an object away with a glowing edge. Used on enemy deaths and magic locks.

Shader	For
Custom/Sprites/Dissolve	Sprites (unlit).
Custom/Dissolve/Lit	3D objects (lit, with shadows).

Shader settings: **Noise Scale** (pattern size), **Edge Width**, and HDR **Inner / Outer Edge** colours.

DissolveEffect

Add it to the object and call `Play()`. It turns off the colliders, animates the dissolve over **Dissolve Duration**, raises `OnDissolveComplete`, and destroys the object if **Destroy On Complete** is on.

The object doesn't need to use a dissolve shader normally. On `Play()`, any material without `_DissolveAmount` is swapped for a copy of **Dissolve Material**, keeping its texture and colour. Leave **Target Renderers** / **Target Colliders** empty to use everything in the object's children.

Enemies automatically play a DissolveEffect in their children when they die. **Door locks** play the one assigned to them.

Shatter

Breaks a sprite into pieces that fly apart and fade. Used when weakpoints are destroyed.

Add **ShatterEffect** and assign a **Shatter Material** using **Custom/Sprites/Shatter** (the sprite's texture is copied in at runtime, so one material works for all sprites).

Setting	What it does
Shatter Duration	Animation length.
Cell Count	Number of pieces.
Grid Resolution	Higher = cleaner piece edges, more triangles.
Randomise Seed	Every instance breaks differently.
Material: **Spread Force**, **Rotation Speed**	How far and fast pieces fly.

Setting	What it does
Material: **Flat Shatter**	Keeps pieces flat to the sprite (2D look).
Material: **Gravity**, **Fade Start / Speed**	Falling and fading.

```
shatter.Play();                         // first sprite in children
shatter.Play(spriteRenderer);           // a specific sprite
shatter.Play(spriteRenderer, false);    // don't destroy this object afterwards
```

WeakPoints use the last form, since they're reused between phases.

> **In code:** a sprite is one flat quad, so `ShatterEffect` builds a new mesh: it grids the sprite, groups triangles into Voronoi cells, and gives each cell its own vertices so pieces move independently. Each vertex stores its cell's centre in **UV2** and a random value in **vertex colour red**, which the shader uses to move and spin each piece. The mesh goes on a new child object, since a GameObject can't have both a SpriteRenderer and a MeshRenderer.

MiddleOutFill (stagger bar)

A UI shader that fills a bar from the **centre outward**. Drop it onto a fill Image's material.

Setting	What it does
Fill Amount	0 to 1.
Edge Skew	Slants the fill edges to match a slanted bar sprite.

`EnemyStagger` sets this per enemy. For a new bar, give each bar its own material copy and set the hidden `_FillUVRect` from the sprite with `DataUtility.GetOuterUV(sprite)` so the fill is measured across the sprite, not its whole atlas.

Danger Volumes

Transparent telegraphs for enemy attacks: a faint fill with bright faces and edges, readable from any angle.

Shader	Used by
Custom/URP/DangerVolumeSphere	AoE telegraphs (`DangerZone`) and the melee hitbox materials.
Custom/URP/DangerVolumeBox	Box-shaped volumes.

Sphere: edges glow using a Fresnel effect. A **ground ring** lights up where the volume meets the floor, showing exactly where the attack lands. The ring needs **Depth Texture** turned on in the URP asset.

Box: Face Thickness and **Edge Thickness** are in world units, so they stay even however the box is scaled. Set **Half Extents** to match the mesh (0.5 for a unit cube).

Both have **Fill / Face / Edge Color** with matching Opacity and Intensity settings.

Volume Profiles

Profile	Used for
GameSceneProfile	Game scenes. Has every custom effect.
TitleSceneProfile	Title scene.
UIProfile	The UI camera's Volume.

Writing to a Volume from code: always use `volume.sharedProfile`. `volume.profile` silently makes a copy, and changes to it never reach the screen. Since the shared profile *is* the asset, Play Mode changes stay on it in the Editor, which is why the runtime scripts reset their values when they start and stop.

Gotchas

- **Feature order on PC_Renderer matters** for effects at the same stage.
- **Only one camera per image with Post Processing on.**
- `sharedProfile`, **not** `profile`.
- `SetFloat`, **not** `SetInt`.
- **Danger Volume ground ring missing?** Turn on Depth Texture.
- **DissolveEffect finishes instantly?** The object has no dissolve shader and no Dissolve Material assigned.